Frontend U1 — Introducción, Tooling moderno

Cátedra: Programación 4 – Frontend

Año: 2026

Profesor: Lucas Ferrero

UTN FACULTAD REGIONAL RAFAELA

SECCIÓN 1 — Introducción

¿Qué es una aplicación web moderna?

En términos generales, no existe una definición unívoca para el concepto de "aplicación web moderna“. Únicamente podemos describir las **características** que una aplicación web moderna puede presentar, las cuales pueden ser:

- **Alojamiento en la nube y escalable**: Gracias al bajo consumo de memoria y al **alto rendimiento**, un mismo servidor puede atender a una mayor cantidad de clientes. “Un alto rendimiento significa que se puede dar servicio a más usuarios desde una misma aplicación utilizando el mismo hardware, lo que reduce aún más la necesidad de invertir en servidores e infraestructura de alojamiento". Por lo tanto, **la escalabilidad se puede manifestar en dos áreas: el rendimiento en cuanto al volumen de clientes y la latencia del servidor al procesar una solicitud web.**

- **Multiplataforma**: Debe ser compatible con entornos Windows, macOS o Unix/Linux. Asimismo, la plataforma de ejecución puede ser de escritorio, web o un dispositivo móvil.

- **Modular y con bajo acoplamiento**: Del mismo modo en que un bloque de LEGO se puede reutilizar para construir distintos objetos, un componente debe poder reutilizarse dentro de la misma aplicación o en una diferente.

- **Soporte para arquitectura tradicional y SPA**: Es compatible tanto con el renderizado en el lado del servidor (SSR, Server-Side Rendering) como con el modelo de aplicación de una sola página (SPA, Single Page Application).

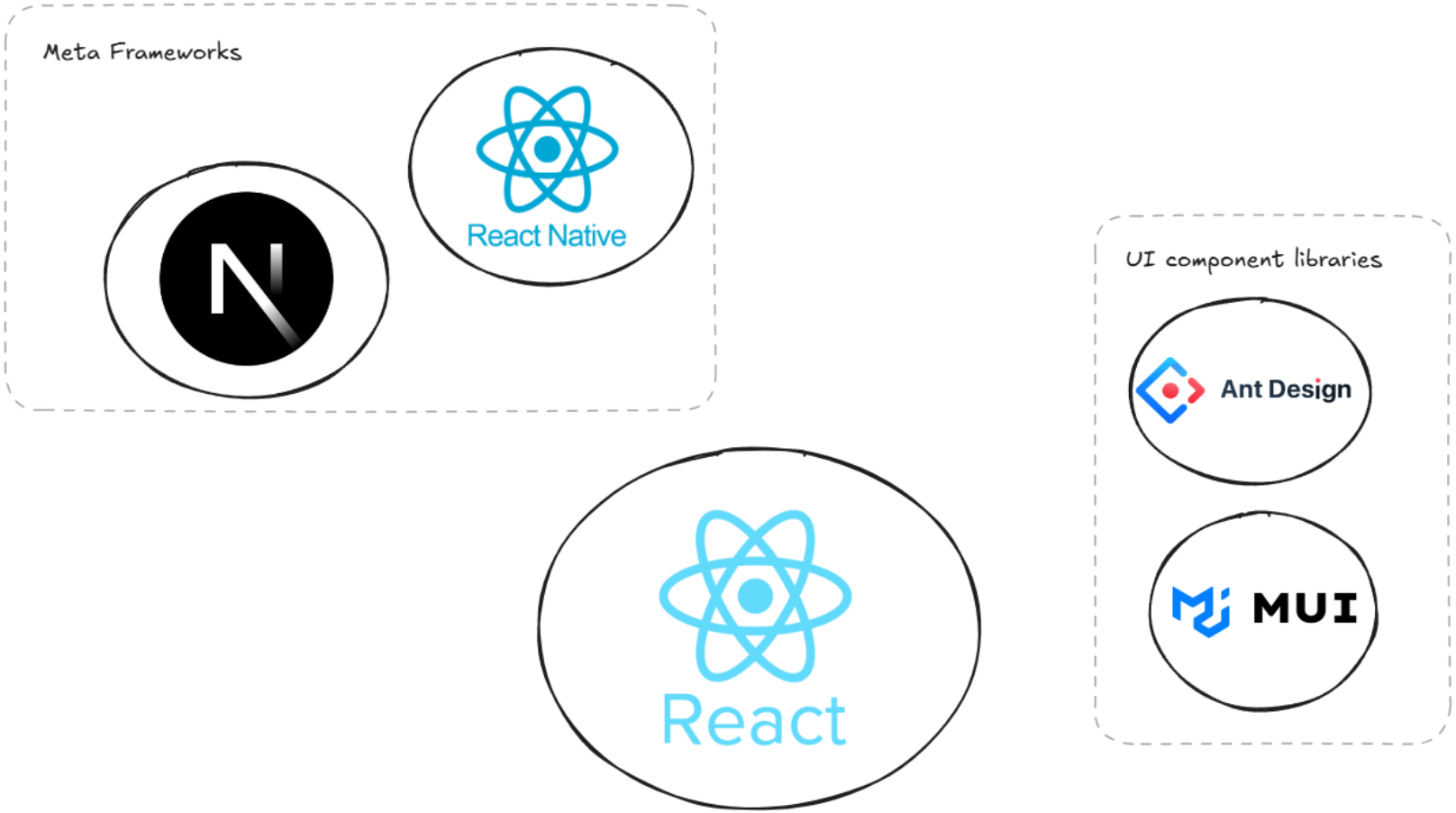
- **Desarrollo e implementación simplificados**: Se debe emplear Integración Continua (CI) y Entrega Continua (CD) para dar soporte al desarrollo progresivo e interativo de la aplicación web.

- **Facilidad de prueba mediante pruebas automatizadas**

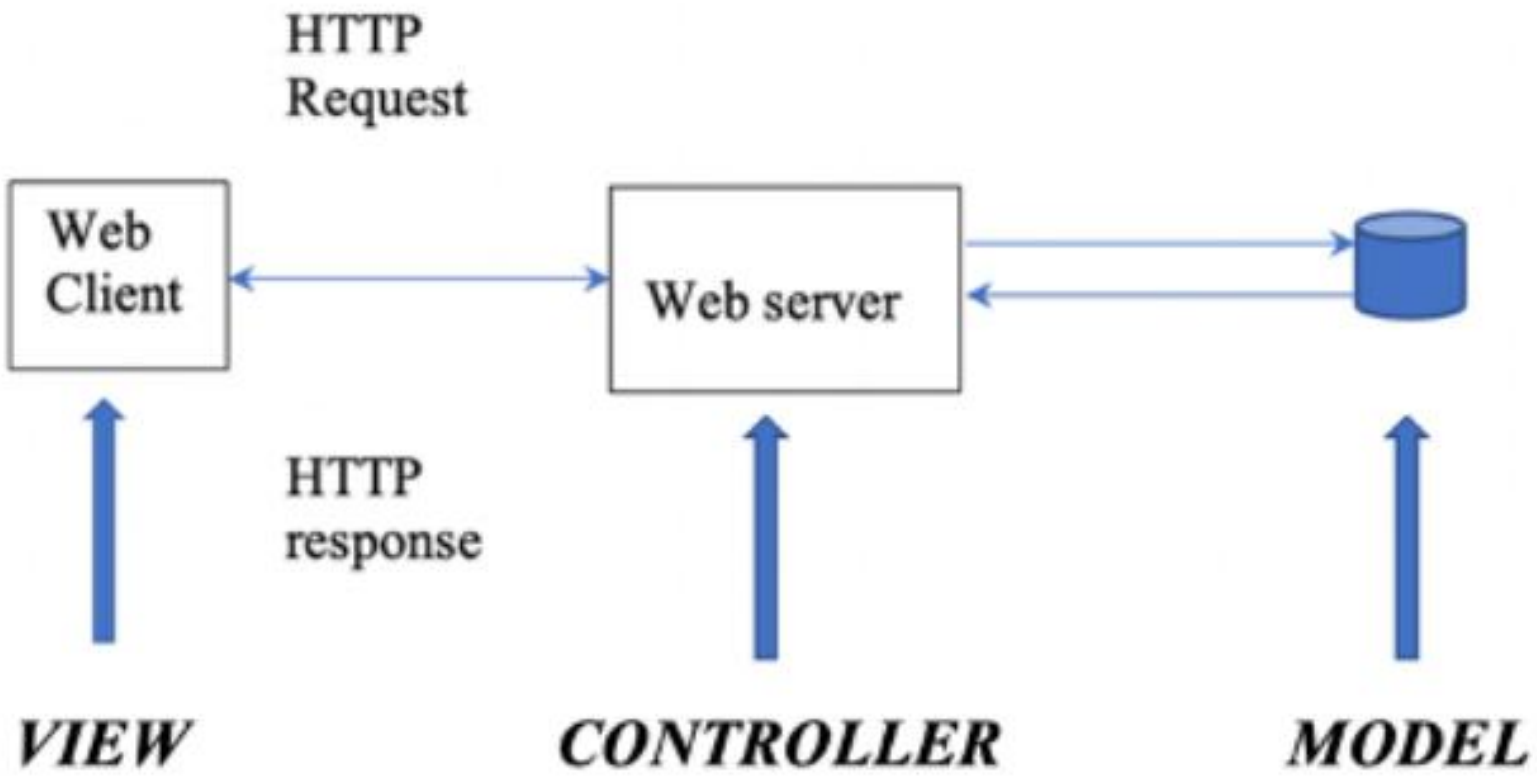
¿Qué es React?

React es una biblioteca de JavaScript para el diseño y desarrollo de interfaces de usuario web. Asimismo, el **ecosistema de React** funciona en conjunto con este para proporcionar **funcionalidades extendidas**. Por ejemplo, React Native se utiliza para el diseño y desarrollo de UI destinadas a dispositivos móviles. Mientras que React trabaja con Node.js para soportar el desarrollo de aplicaciones de una sola página (SPA), Next.js utiliza a React en el front-end para dar soporte a aplicaciones web con renderizado en el lado del servidor (SSR). Por lo tanto, Next.js se considera parte del ecosistema de React.

Ecosistema React



La letra "V" en el patrón MVC (Modelo-Vista-Controlador) está relacionada con la interfaz de usuario (UI) de una aplicación web. De igual manera, nos referimos a la interfaz de usuario de una aplicación web como la interfaz (UI) del frontend.



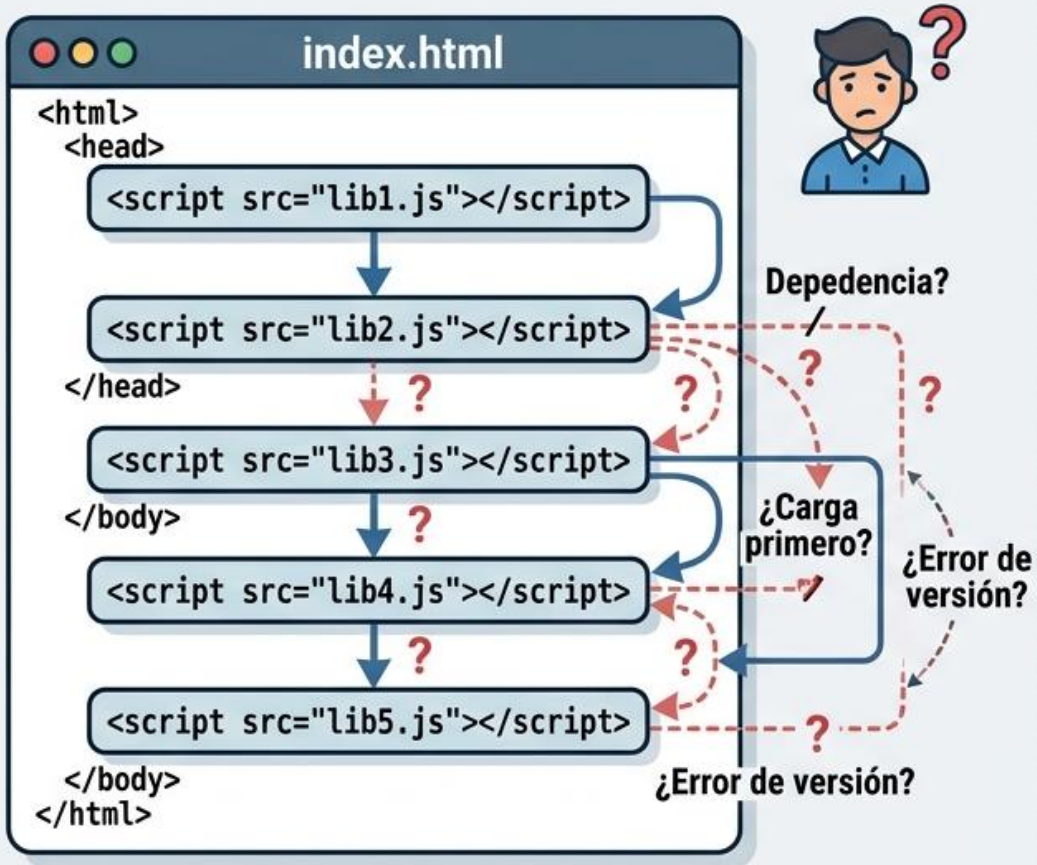
SECCIÓN 2 — npm y gestión de dependencias

De incluir scripts a construir una aplicación

En Programación 3 construimos interfaces con HTML, CSS y JavaScript enlazando archivos <script> directamente en el HTML, en un orden que nosotros mismos definíamos manualmente. Esta forma de trabajar es perfectamente válida para páginas pequeñas, pero conviene detenernos a pensar qué ocurre cuando el proyecto crece: una aplicación de gestión de pedidos, por ejemplo, con módulos de productos, clientes, carrito y facturación.

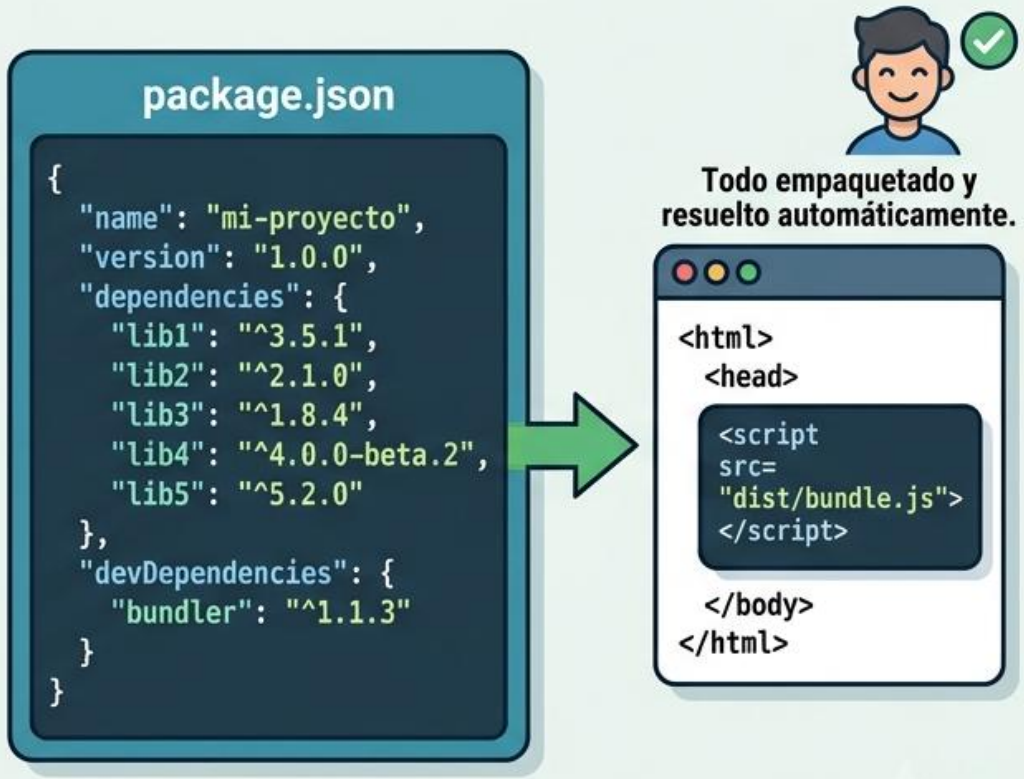
Evidentemente utilizar scripts para aplicaciones medianas/grandes se vuelve tedioso, ya que como desarrolladores administrar archivos .js propios y de terceros de forma manual hace lento el avance del proyecto y el mismo se vuelve propenso a errores.

SITUACIÓN TRADICIONAL: CAOS EN EL CLIENTE



Traditional Script Hell (HTML & Cross-Dependency)

SOLUCIÓN MODERNA: GESTIÓN CENTRALIZADA (Node.js/npm)



Modern Solution: Organized `package.json`

Npm

npm (Node Package Manager) es un gestor de dependencias y orquestador de tareas, posee:

- Un registro público de paquetes (el npm registry) donde cualquier desarrollador puede publicar y cualquier otro puede descargar código reutilizable.
- Un archivo de manifiesto, *package.json*, que declara de forma explícita y legible qué dependencias necesita el proyecto, en qué versiones, y qué comandos (scripts) se pueden ejecutar.
- Un archivo de lockfile, package-lock.json, que fija las versiones exactas instaladas — no solo las declaradas — garantizando que el mismo proyecto se instale de forma idéntica en cualquier máquina.

Anatomía de package.json

Ejemplo package.json de un proyecto Frontend:

```json
{
  "name": "gestion-pedidos-frontend",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview",
    "lint": "eslint ."
  },
  "dependencies": {
    "react": "^19.1.0",
    "react-dom": "^19.1.0",
    "axios": "^1.7.9"
  },
  "devDependencies": {
    "@types/react": "^19.1.0",
    "@types/react-dom": "^19.1.0",
    "typescript": "^5.7.3",
    "vite": "^6.0.7"
  },
  "engines": {
    "node": ">=20.0.0"
  }
}
```

Existen dos categorías de dependencias:

* **dependencies**: paquetes necesarios para que la aplicación funcione en producción — React, Axios, cualquier librería que se va a ejecutar en el navegador del usuario

- **devDependencies**: paquetes necesarios solo durante el desarrollo — TypeScript (que se compila a JS), Vite, linters, tipos (@types/*). Ninguno de estos termina en el bundle que descarga el usuario final.

La herramienta de compilación utilizada, deja afuera del *bundle final* todo lo que está en *devDependencies*, precisamente porque asume que ese código no es necesario tenerlo en la aplicación compilada final.

Dependencias en un proyecto Frontend (React)

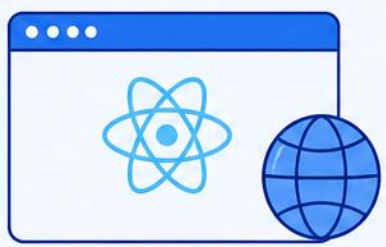
Dependencies

viaja al navegador

Son las librerías que necesita tu aplicación para funcionar en producción.

Ejemplos:

- react
- react-dom
- react-router-dom
- axios
- antd
- etc.



Se incluye en el bundle final
y viaja al navegador del usuario.

DevDependencies

se queda en la máquina del desarrollador

Son las herramientas que solo se usan durante el desarrollo y construcción del proyecto.

Ejemplos:

- vite
- typescript
- eslint
- prettier
- @types/react
- etc.



No se incluye en el bundle final
y no viaja al navegador.

Instalación, versionado semántico y node_modules

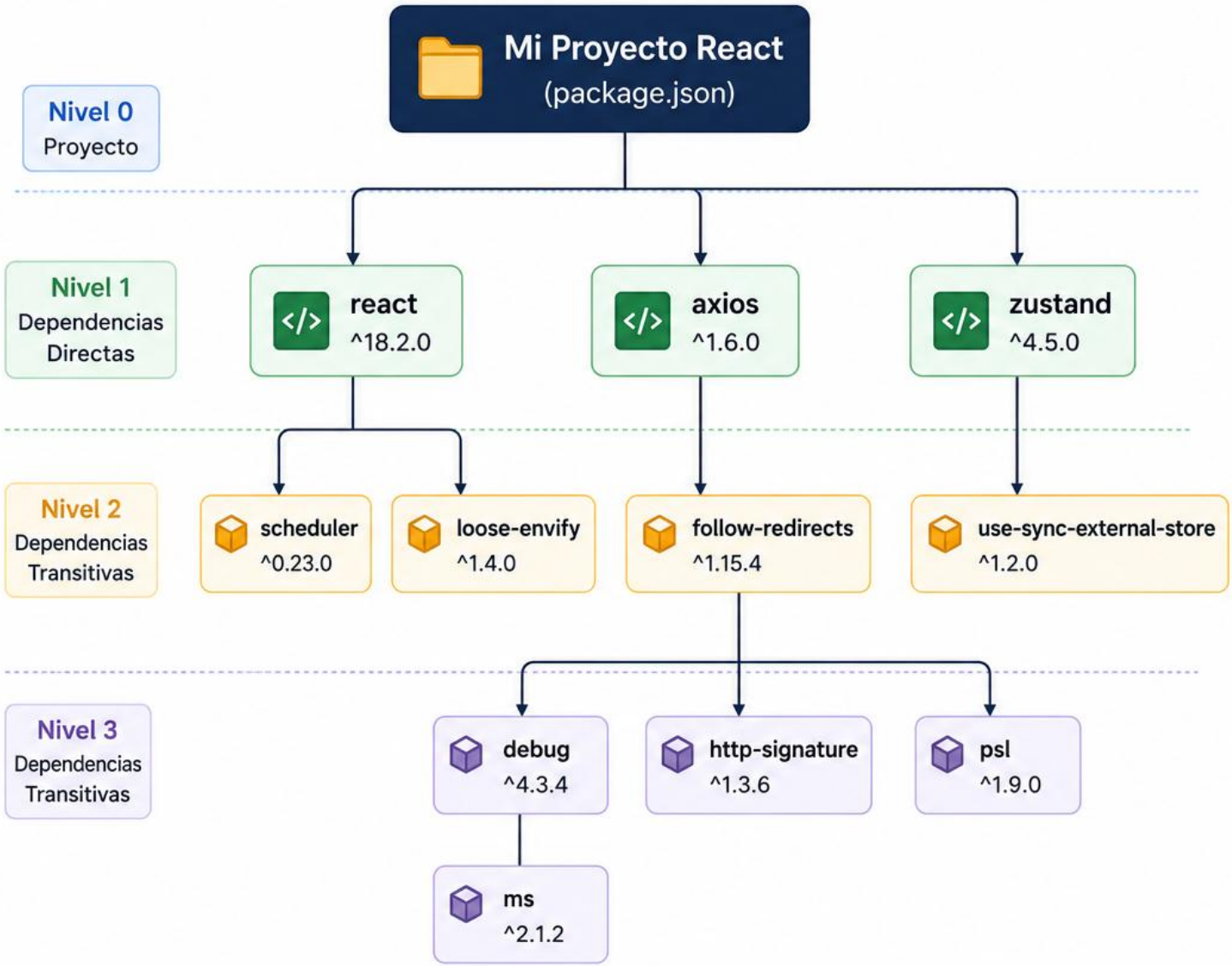
Cuando ejecutamos *npm install*, ocurre un proceso que nos conviene entender:

1. *npm* lee *package.json* y, si existe package-lock.json, prioriza las versiones exactas de este último archivo.
2. Se resuelve el árbol completo de dependencias — las dependencias de nuestras dependencias (y las de esas, recursivamente).
3. Descarga cada paquete del registry y lo coloca en node_modules/.
4. Escribe o actualiza package-lock.json reflejando exactamente qué se instaló.

El símbolo ^ que aparece delante de las versiones en *package.json* (por ejemplo *"axios": "^1.7.9"*) indica un rango permitido: acepta actualizaciones de MINOR y PATCH, pero nunca de MAJOR. Es decir, *^1.7.9* permite instalar *1.8.0* o *1.7.10*, pero nunca *2.0.0*

Árbol de Dependencias en un Proyecto React

Las dependencias directas que agregas a tu proyecto pueden traer sus propias dependencias (transitivas).



Proyecto

Es tu aplicación React.
Define las dependencias que necesitas directamente.

Dependencias Directas

Las que agregas explícitamente en tu package.json
(ej: **react**, **axios**, **zustand**).

Dependencias Transitivas

Son dependencias que necesitan tus dependencias directas para funcionar.
No las agregas vos, pero se instalan automáticamente.

Importante

Cada vez que instalas una dependencia directa, puede traer muchas otras consigo. Por eso el árbol puede crecer.

Scripts de npm

Los scripts de *package.json* son comandos para ejecutar tareas repetitivas. Por ejemplo: iniciar la aplicación, hacer build para producción, ejecutar tests, etc. Nos sirven como atajos para no escribir textos largos en la consola.



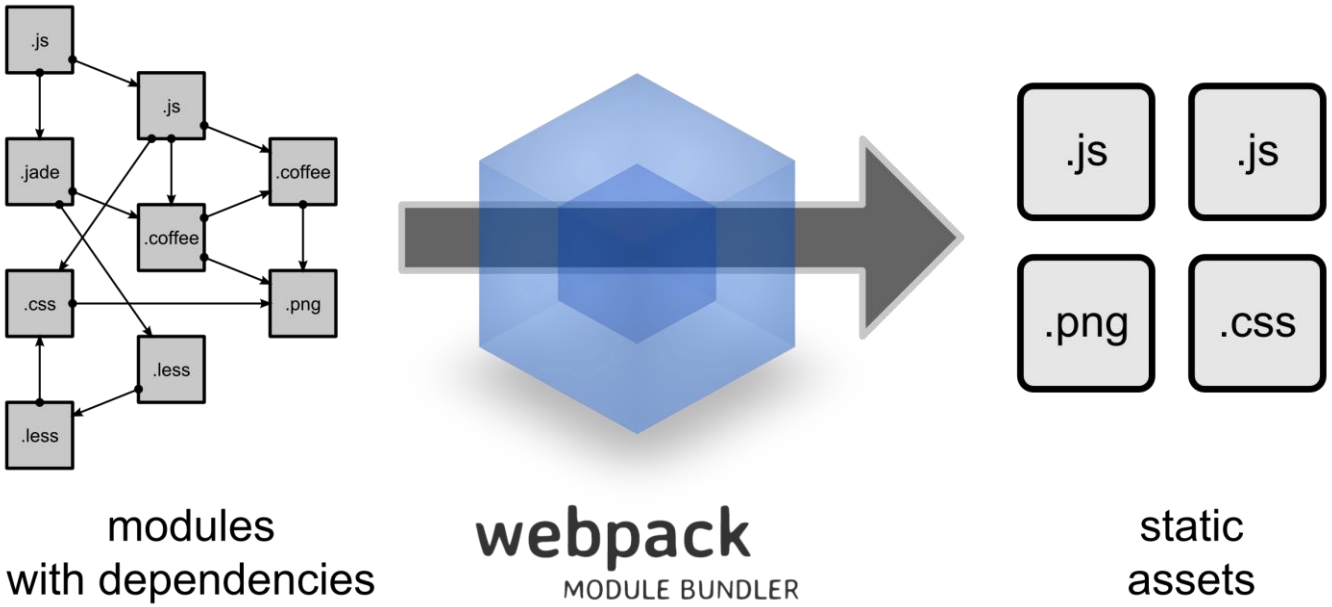
¿Para qué sirven?

- Ahorrar tiempo: Agrupan instrucciones largas y difíciles de recordar en una sola palabra simple.
- Compartir reglas: Ayudan a que todo el grupo de trabajo use exactamente el mismo comando y los mismos datos en el proyecto.
- Automatizar procesos: Sirven para compilar archivos, actualizar bases de datos o ejecutar herramientas de linting.

SECCIÓN 3 — Module bundlers y Vite

Module Bundler

Un Module Bundler (empaquetador de módulos) es una herramienta de desarrollo que toma todos los archivos de código fuente de un proyecto web (JavaScript, CSS, imágenes, TypeScript, JSX, etc.), analiza cómo se relacionan entre sí y los procesa para unificarlos en un conjunto reducido de archivos optimizados que el navegador puede entender y cargar eficientemente.



Proceso de empaquetado: de un grafo de dependencias a archivos estáticos

¿Qué problema resuelve?

Sin module bundler	Con module bundler
Decenas de peticiones HTTP: El navegador tiene que descargar 30 o 50 archivos .js por separado, volviendo la carga muy lenta.	Pocas peticiones: El proyecto se reduce a 1 o 2 archivos .js minificados.
Colisión de variables globales: Si dos archivos declaran la misma variable *const usuario*, se sobreescriben.	Encapsulamiento: Cada archivo funciona como un módulo aislado con dependencias explícitas (import / export).
Incompatibilidad de sintaxis: El código moderno (ES6+) no corre en todos los navegadores.	Transpilación automática: Convierte TypeScript o JS moderno a JS compatible en el proceso de empaquetado

Empaquetadores de módulos más utilizados:

- Webpack
- Vite
- Rollup

Las 3 tareas clave que realiza

1. **Construye el Grafo de Dependencias**: Empieza desde un archivo raíz (por ejemplo, index.js) y sigue cada declaración *import* o *require* para mapear exactamente qué archivo depende de cuál otro.
2. **Transforma y Procesa**:
   - Convierte TypeScript a JavaScript puro.
   - Transforma JSX (React) a funciones de JS.
   - Compila sintaxis como SASS/LESS a CSS plano.
   - Optimiza e incrusta imágenes.
3. **Optimización y Salida (Bundling)**:
   - **Minificación**: Elimina espacios en blanco, comentarios y acorta nombres de variables.
   - **Tree Shaking**: “Sacude el árbol" para eliminar las partes de código importadas que nunca se llegan a ejecutar.
   - **Code Splitting**: Divide el código en partes más pequeñas que se cargan solo cuando el usuario entra a una ruta específica de la aplicación.

Vite

- Vite es una herramienta moderna para el desarrollo de aplicaciones Frontend
- Trata de forma diferenciada dos momentos del ciclo de vida de un proyecto:
  - En desarrollo: Vite utiliza una técnica llamada HMR (Hot Module Replacement) para que el flujo de desarrollo sea más ágil.
  - En producción: Genera un output lo más optimizado posible priorizando la calidad del artefacto final por sobre la velocidad de construcción, ya que el build de producción se ejecuta una sola vez por despliegue, no en cada guardado de archivo.

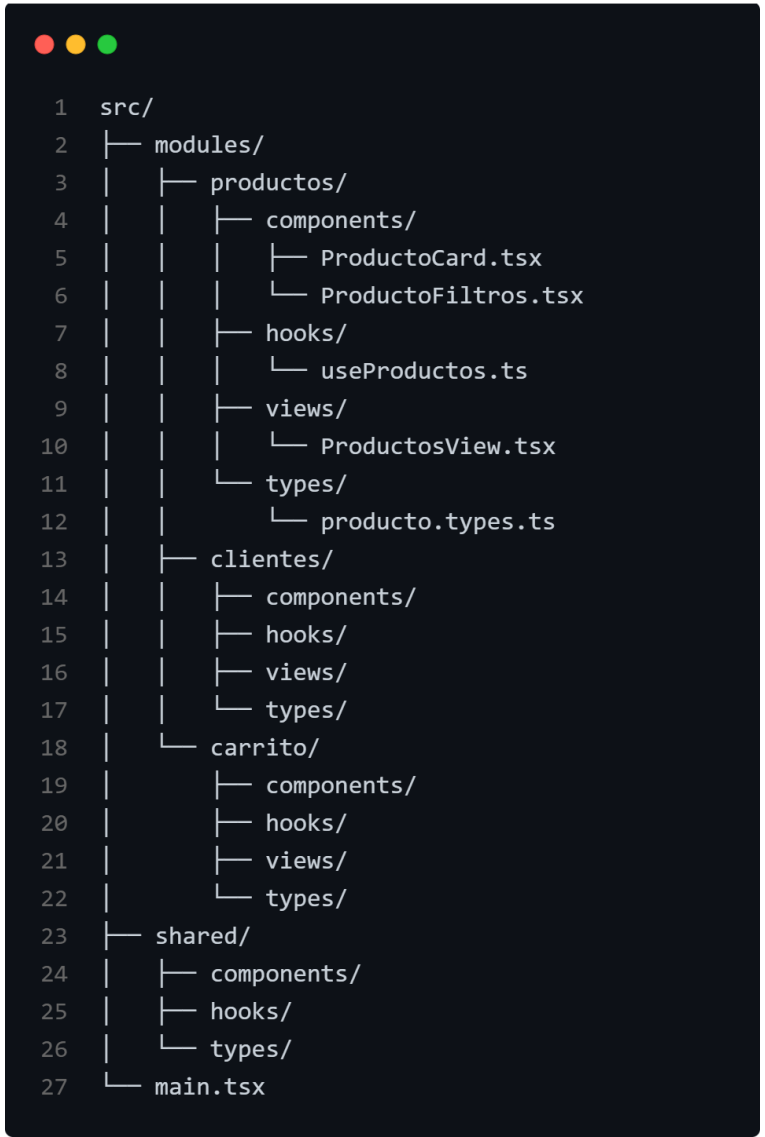
- Instalar Vite y hacer scaffolding con react: https://vite.dev/guide/

- El archivo vite.config.ts se utiliza para configurar distintos aspectos de Vite
- alias: configurar @ como un atajo hacia src/ es una práctica recomendada que evita rutas relativas como *../../../modules/productos/types* a la hora de importar distintos artefactos. Con el alias configurado, ese mismo import se escribe @/modules/productos/types
- Al ejecutar *npm run build*, Vite genera una carpeta dist/ con los archivos finales: JavaScript y CSS minificados, con nombres que incluyen un hash de contenido (por ejemplo index-a1b2c3.js) que permite un cacheo agresivo por parte del navegador — si el contenido no cambia, el hash no cambia, y el navegador puede reutilizar la versión cacheada indefinidamente.

SECCIÓN 4 — Arquitectura de archivos y carpetas

Feature-based architecture

Se trata de agrupar el código por feature de negocio. Cada feature (o característica) vive en su propia carpeta dentro de /src/modules, con una subestructura interna consistente:



Cada subcarpeta cumple un rol específico:

- **components/:** piezas de UI reutilizables dentro de ese feature (una tarjeta de producto, un formulario de filtros).
- **hooks/**: lógica reutilizable con estado, encapsulada en custom hooks
- **views/**: componentes de "página completa" que ensamblan components + hooks para ese feature, y que típicamente se conectan directamente a una ruta (React Router).
- **types/**: las interfaces y types específicos de ese dominio — por ejemplo, la interfaz Producto.
- **services/**: llamadas a APIs externas.

Convenciones de nombres

Es importante mantener consistencia en cómo nombramos los archivos, ya que esto permite identificar de un vistazo qué tipo de artefacto estamos mirando:

Tipo de archivo	Convención	Ejemplo
Componentes	PascalCase	ProductoCard.tsx
Custom hooks	camelCase, prefijo use	useProductos.ts
Vistas	PascalCase, sufijo View	ProductosView.tsx
Types/interfaces	camelCase, sufijo .types	producto.types.ts
Utilidades	camelCase	*stringFormat.ts*


UTN
FACULTAD REGIONAL
RAFAELA